

Practica Patrón Strategy

Víctor Manuel Peiró Martínez

FIIS 2ºA

Diseño De Software

Contexto

Estamos modelando una guardería que contiene 2 doctores. El Doctor Wang es un Oculista que examina el día 15 de cada mes. El Doctor Fong es un Logopeda que examina el día 28 de cada mes. Cada médico debe hacer tres cosas:

1. Examinar a los niños de la guardería
2. Enviar los resultados a los padres de esos niños
3. Enviar la factura

Para realizar esto se me ocurren 2 posibles soluciones aplicando el patrón de diseño strategy. La primera solución contendrá un método para cada una de estas tareas. La segunda manera es que creamos una subestrategia que sea capaz de ejecutar las tres a la vez para que cada vez que se examine un niño, se envíe también el resultado y la factura.

Solución 1

Información:

Esta solución es la primera solución descrita donde definimos las 3 tareas y se ejecutan por separado.

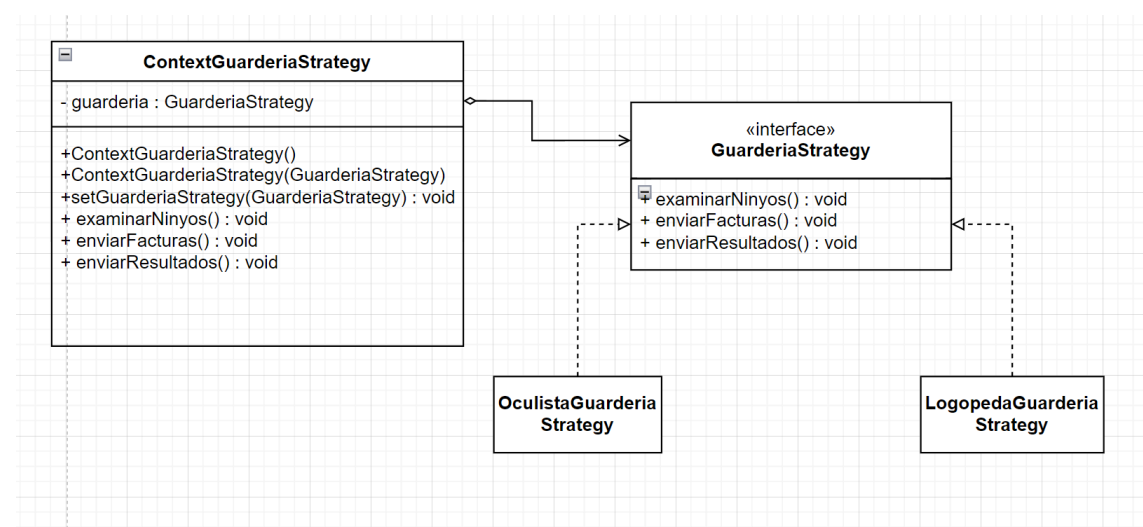
Estrategia: Es la interfaz común con los métodos que van a ser definidos en las estrategias concretas. En este caso es la interfaz **GuarderiaStrategy**

Contexto: Está compuesto por un objeto de tipo Strategy e instanciado con una estrategia concreta. En este caso es la clase **ContextGuarderiaStrategy**

Estrategias Concretas: Define la implementación de cada algoritmo mediante la estrategia. En esta solución tenemos 2 estrategias concretas.

- **OculistaGuarderiaStrategy**
- **LogopedaGuarderiaStrategy**

Diagrama UML



Como podemos ver, se mantiene la relación de agregación entre el contexto y la estrategia.

Código:

Interfaz Estrategia:

```
package EntregaStrategy;
//Estrategia Metodos a definir
public interface GuarderiaStrategy {
    public abstract void examinarNinos();
    public abstract void enviarFacturas();
    public abstract void enviarResultados();
}
```

Estrategias Concretas:

```
package EntregaStrategy;
//Estrategia Concreta para el Oculista --> Se define los metodos de la estrategia
class OculistaGuarderiaStrategy implements GuarderiaStrategy{

    public void examinarNinos() {
        System.out.println("El Oculista esta examinando a la vision niños");
    }
    public void enviarFacturas() {
        System.out.println("El Oculista envia las facturas");
    }
    public void enviarResultados() {
        System.out.println("El Oculista te ha enviado los resultados del examen");
    }
}
```

```
package EntregaStrategy;
//Estrategia Concreta para el Logopeda --> Se define los metodos de la estrategia
public class LogopedaGuarderiaStrategy implements GuarderiaStrategy{
    public void examinarNinos() {
        System.out.println("El Logopeda esta examinando el habla de los niños");
    }
    public void enviarFacturas() {
        System.out.println("El Logopeda te envia las facturas");
    }
    public void enviarResultados() {
        System.out.println("El Logopeda te envia los resultados del examen");
    }
}
```

Contexto:

```
package EntregaStrategy;
//Contexto que utiliza el patron
public class ContextGuarderiaStrategy implements GuarderiaStrategy{

    private GuarderiaStrategy guarderiaStrategy; //Relacion de Agregacion

    //Constructor por defecto --> Asigna al Oculista como Estrategia por Defecto
    public ContextGuarderiaStrategy() {
        this(new OculistaGuarderiaStrategy());
    }

    //Constructor principal
    public ContextGuarderiaStrategy(GuarderiaStrategy guarderiaStrategy) {
        this.guarderiaStrategy = guarderiaStrategy;
    }

    //Setter --> Permite cambiar de estrategia
    public void setGuarderiaStrategy(GuarderiaStrategy guarderiaStrategy) {
        this.guarderiaStrategy = guarderiaStrategy;
    }

    //Metodos definidos en la estrategia --> Permite su ejecucion
    public void examinarNinos() {
        this.guarderiaStrategy.examinarNinos();
    }

    public void enviarFacturas() {
        this.guarderiaStrategy.enviarFacturas();
    }

    public void enviarResultados() {
        this.guarderiaStrategy.enviarResultados();
    }
}
```

Test:

```
package EntregaStrategy;
public class GuarderiaTest {
    public static void main(String[] args) {
        ContextGuarderiaStrategy guarderia = new ContextGuarderiaStrategy();
        System.out.println("Dia 15:");
        System.out.println("Doctor Wang (Oculista): ");
        guarderia.setGuarderiaStrategy(new OculistaGuarderiaStrategy());
        guarderia.examinarNinos();
        guarderia.enviarFacturas();
        guarderia.enviarResultados();

        System.out.println("Dia 28:");
        System.out.println("Doctor Fong (Logopeda): ");
        guarderia.setGuarderiaStrategy(new LogopedaGuarderiaStrategy());
    }
}
```

```
        guarderia.examinarNinos();
        guarderia.enviarFacturas();
        guarderia.enviarResultados();
    }
}
```

Resultado

```
<terminated> GuarderiaTest [Java Application] C:\Users\Victor Peiro\AppData\Local\Temp\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_22.0.2.v20240802-1626\jre\bin
Dia 15:
Doctor Wang (Oculista):
El Oculista esta examinando a la vision niños
El Oculista envia las facturas
El Oculista te ha enviado los resultados del examen
Dia 28:
Doctor Fong (Logopeda):
El Logopeda esta examinando el habla de los niños
El Logopeda te envia las facturas
El Logopeda te envia los resultados del examen
```

Solución 2

Información:

Añadimos una nueva estrategia que va a ser la responsable de unificar los 3 métodos y ejecutarlos a la vez.

Estrategia: Es la interfaz común con los métodos que van a ser definidos en las estrategias concretas. En este caso son las interfaces:

- **GuarderiaMedicalStrategy**
- **MedicalStrategy**

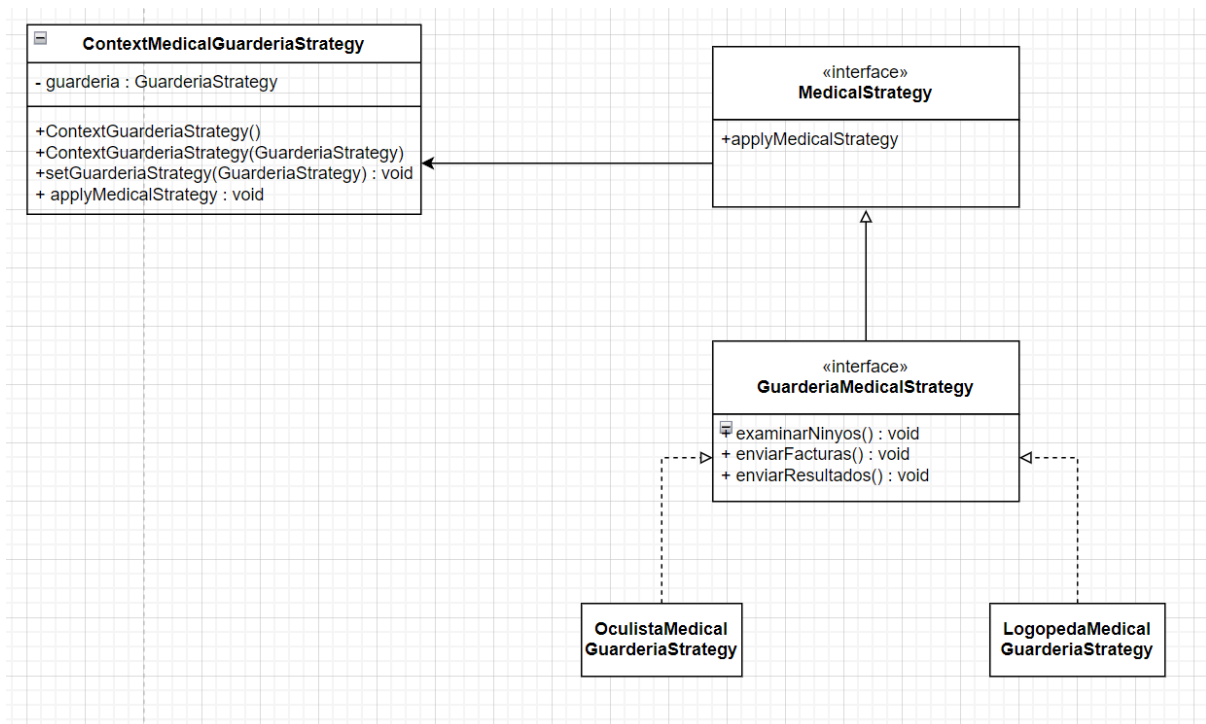
Contexto: Está compuesto por un objeto de tipo Strategy e instanciado con una estrategia concreta.

En este caso es la clase **ContextMedicalGuarderiaStrategy**

Estrategias Concretas: Define la implementación de cada algoritmo mediante la estrategia. En esta solución tenemos 2 estrategias concretas.

- **OculistaMedicalGuarderiaStrategy**
- **LogopedaMedicalGuarderiaStrategy**

Diagrama UML



Codigo:

Estrategias:

```
package EntregaStrategy2;
//Subestrategia que sirve para unificar
public interface MedicalStrategy {
    public abstract void applyMedicalStrategy();
}
```

```
package EntregaStrategy2;
//Estrategia principal que contiene el nuevo metodo
public interface GuarderiaMedicalStrategy extends MedicalStrategy {
    public abstract void applyMedicalStrategy();
    public abstract void examinarNinys();
    public abstract void enviarResultados();
    public abstract void enviarFacturas();
}
```

Estrategias Concretas

```
package EntregaStrategy2;
//Estrategia Concreta para el Logopeda
public class LogopedaMedicalGuarderiaStrategy implements GuarderiaMedicalStrategy {

    public void examinarNinys() {
        System.out.println("El Logopeda esta examindando el habla de los niños");
    }
    public void enviarFacturas() {
        System.out.println("El Logopeda envia las facturas");
    }
    public void enviarResultados() {
        System.out.println("El Logopeda te ha enviado los resultados del examen");
    }
    public void applyMedicalStrategy() {
        this.examinarNinys();
        this.enviarResultados();
        this.enviarFacturas();
    }
}
```

```
package EntregaStrategy2;
//Estrategia Concreta para un Oculista
public class OculistaMedicalGuarderiaStrategy implements GuarderiaMedicalStrategy {

    public void examinarNinys() {
        System.out.println("El Oculista esta examindando a la vision niños");
    }
    public void enviarFacturas() {
        System.out.println("El Oculista envia las facturas");
    }
    public void enviarResultados() {
```

```

        System.out.println("El Oculista te ha enviado los resultados del examen");
    }
    public void applyMedicalStrategy() {
        this.examinarNinos();
        this.enviarResultados();
        this.enviarFacturas();
    }
}

```

Contexto:

```

package EntregaStrategy2;
//Contexto que contiene la agregacion
public class ContextMedicalGuarderiaStrategy implements MedicalStrategy{

    private MedicalStrategy medicalStrategy;

    public ContextMedicalGuarderiaStrategy() {
        this(new OculistaMedicalGuarderiaStrategy());
    }

    public ContextMedicalGuarderiaStrategy(MedicalStrategy medicalStrategy) {
        this.medicalStrategy = medicalStrategy;
    }

    public void setMedicalStrategy(MedicalStrategy medicalStrategy) {
        this.medicalStrategy = medicalStrategy;
    }

    //Aplica la estrategia
    public void applyMedicalStrategy() {
        this.medicalStrategy.applyMedicalStrategy();
    }
}

```

Test:

```

package EntregaStrategy2;
public class MedicalGuarderiaTest {
    public static void main(String[] args) {
        ContextMedicalGuarderiaStrategy guarderia = new ContextMedicalGuarderiaStrategy();
        System.out.println("Dia 15:");
        System.out.println("Dr. Wang: ");
        guarderia.setMedicalStrategy(new OculistaMedicalGuarderiaStrategy());
        guarderia.applyMedicalStrategy();

        System.out.println("Dia 28: ");
        System.out.println("Dr. Fong");
        guarderia.setMedicalStrategy(new LogopedaMedicalGuarderiaStrategy());
        guarderia.applyMedicalStrategy();
    }
}

```



```
}  
}
```

Resultado

```
<terminated> MedicalGuarderiaTest [Java Application] C:\Users\Victor Peiro\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_22.0  
Dia 15:  
Dr. Wang:  
El Oculista esta examinando a la vision niños  
El Oculista te ha enviado los resultados del examen  
El Oculista envia las facturas  
Dia 28:  
Dr. Fong  
El Logopeda esta examinando el habla de los niños  
El Logopeda te ha enviado los resultados del examen  
El Logopeda envia las facturas
```